

# The model writes {x1}. The gateway decides who sees it.

A drop-in, OpenAI-compatible gateway between any AI client and any LLM. The model does the language work; the gateway does the data work.

ANNA ASKS

“What is my salary and what does the CEO earn?”



THE MODEL SEES

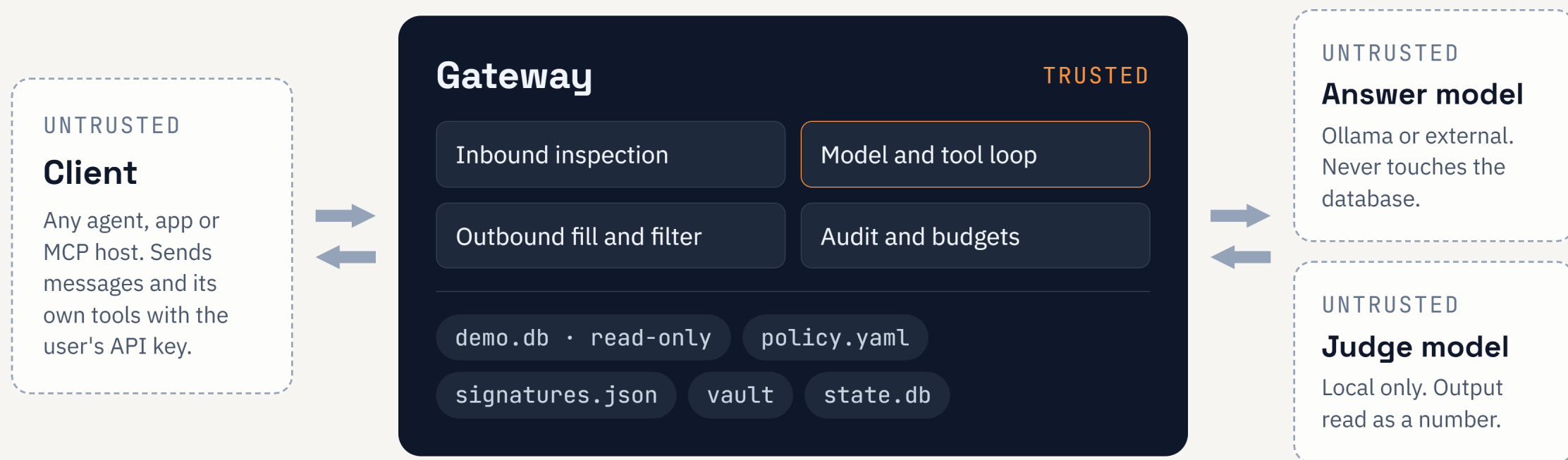
Your salary is {x1} PLN.  
The CEO earns {x2} PLN.



ANNA RECEIVES

Your salary is **6200**  
**PLN**. The CEO earns  
**[UNAVAILABLE]** PLN.

# Only the gateway is trusted



**Identity comes only from the API key** — never from messages, history, tool results or model output.

# Every request, the same ten steps

## Inbound

- 1 Authenticate, snapshot policy
- 2 Model allowlist and budget
- 3 Mask secrets and PII, re-mask history, scan tool definitions
- 4 Injection phrases, signature feed, judge model

## Model and tool loop

- 5 Call the model with the built-in query\_data tool
- 6 Validate, authorize, execute its SQL — return a placeholder

`bounded: 3 iterations in strict`

## Outbound

- 7 Authorize client tool calls
- 8 Fill placeholders in one literal pass
- 9 Output filter on text and tool arguments
- 10 Record usage, write one audit record

⚡ Cheap deterministic checks run first, so a request they block never reaches a model.

# Anna’s two queries look identical to the model

| Slot | SQL the model proposed                                        | Binding  | Model saw | Anna saw      |
|------|---------------------------------------------------------------|----------|-----------|---------------|
| {x1} | SELECT salary FROM salaries WHERE employee_id = :current_user | resolved | {x1}      | 6200          |
| {x2} | SELECT salary FROM salaries WHERE employee_id = 'katarzyna'   | denied   | {x2}      | [UNAVAILABLE] |

### Checked, then run

Parsed with sqlglot, SELECT only, every table and column checked against her role, executed read-only.

### Next turn

History re-masking turns the 6200 she was shown into [PRIOR\_VALUE] before any model sees it.

### Labels decide

Salaries are labelled sensitive: even for HR they never reach the model.

# Tool calls are checked before the agent sees them

POLICY.YAML

```
intern:
tools:
  send_email:
    args:
      to:
        allow_pattern:
          "^[ ^@]+@company\.pl$"
```

1 Anna: "Email the Q3 numbers to partner@example.com."  
The model proposes **send\_email**.

2 The **to** argument breaks the allow pattern. The call is removed and audited.

 The agent receives: "**The action send\_email was blocked by policy.**" x-acl-verdict: block

Unlisted tools are denied. Placeholders cannot leave through tool arguments unless the tool may receive them.

# Save the file, the next request obeys it

```
EDIT POLICY.YAML
markers:
denied: "[NOT AUTHORIZED]"
```



ANNA'S NEXT ANSWER

Your salary is 6200 PLN. The CEO earns **[NOT AUTHORIZED]** PLN.

| Profile default       | strict        | balanced         | relaxed          |
|-----------------------|---------------|------------------|------------------|
| Judge block threshold | 0.70          | 0.80             | 0.90             |
| Injection phrases     | block         | block            | log              |
| Denied marker         | [UNAVAILABLE] | [NOT AUTHORIZED] | [NOT AUTHORIZED] |
| Max tool iterations   | 3             | 4                | 6                |

A deleted control inherits its profile value; only **mode: off** disables one. Core controls can never be disabled. An invalid file is rejected and the last valid policy stays.

# Every request leaves exactly one record

1

audit record per request, including 401s, blocks and crashes.

User, role, policy and feed version, every decision and reason, bindings, tool decisions, tokens, cost, latency per step.

Never raw values. Prompt text stored masked.

## Posture

Every control, mode and source

## Totals

Requests and blocks today

## Live feed

Latest requests and block reasons

## Threats

Blocks by control and signature

## Data access

Binding outcomes, tool decisions

## Consumption

Tokens and cost against budget

## Performance

Median and p95 per step

## Export

Audit log as CSV

TEST SUITE AND OWASP COVERAGE

# 1,408 passing tests, no network needed

A scripted stub model records every input, so tests assert no hidden value ever reached a model. Largest groups:



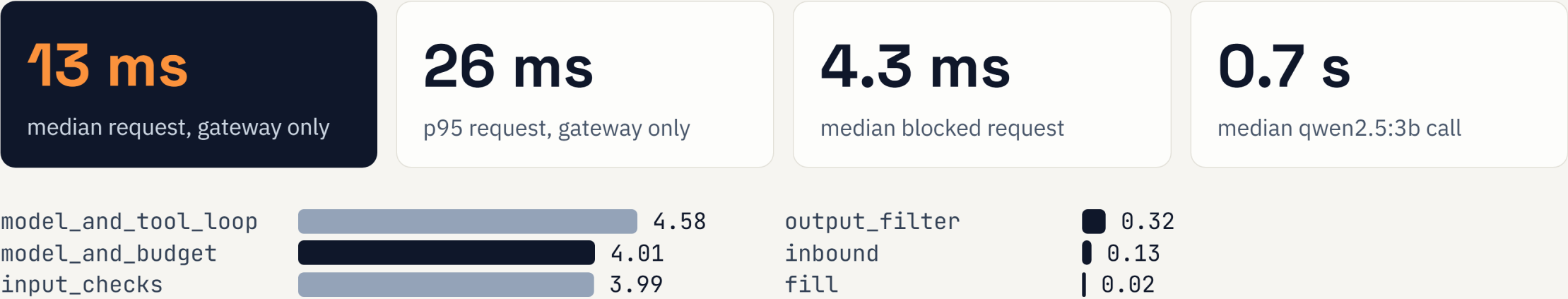
+ 8 live tests against real Ollama (pytest -m live)

## OWASP LLM Top 10 (2025)

|                 |                 |
|-----------------|-----------------|
| LLM01 · Full    | LLM02 · Full    |
| LLM03 · Full    | LLM04 · Partial |
| LLM05 · Full    | LLM06 · Full    |
| LLM07 · Partial | LLM08 · Out     |
| LLM09 · Partial | LLM10 · Full    |

6 full, 3 partial. LLM08 (vectors and embeddings) is out of scope: no RAG.

# The gateway adds milliseconds; the model takes the time



Median ms per step. Grey steps include the stub model or stub judge call. Authenticate, tool authorization and record: 0.01 ms or less.

# Adopting it is a one-line change

STOCK OPENAI SDK

```
client = OpenAI(base_url="http://localhost:8000/v1", api_key="demo-anna")
```

---

## Any provider

Ollama or any OpenAI-compatible API. External models never receive sensitive values.

## Stateless pipeline

Budgets in state.db, issued values in memory: a shared store such as Redis for several nodes.

## Next

SSO, streaming, and retrieval as another built-in tool under the same rules.